INFORMATICA · SISTEMI DI NUMERAZIONE

# Oltre il binario: l'esadecimale

Sedici cifre per scrivere i bit in modo corto e leggibile: colori, indirizzi di rete, memoria.

IL PROBLEMA

# 24 cifre binarie o 6 esadecimali?

Lo stesso numero in binario

```
1101 0010 0011 1111 1010 1100
```

... e in esadecimale

```
D 2 3 F A C
```

Il binario è lungo e facile da sbagliare. Il decimale è corto ma non "si allinea" ai bit. L'esadecimale è corto **e** si legge a gruppi di 4 bit.

BASE 16

# Sedici cifre: da 0 a 9, poi da A a F

| Decimale | Binario | Esadecimale |
|---|---|---|
| 0 | 0000 | 0 |
| 1 | 0001 | 1 |
| 2 | 0010 | 2 |
| 3 | 0011 | 3 |
| 4 | 0100 | 4 |
| 5 | 0101 | 5 |
| 6 | 0110 | 6 |
| 7 | 0111 | 7 |

| Decimale | Binario | Esadecimale |
|---|---|---|
| 8 | 1000 | 8 |
| 9 | 1001 | 9 |
| 10 | 1010 | A |
| 11 | 1011 | B |
| 12 | 1100 | C |
| 13 | 1101 | D |
| 14 | 1110 | E |
| 15 | 1111 | F |

PERCHÉ PROPRIO 16

# $16 = 2^4$: una cifra esadecimale vale 4 bit

## 4 bit

### 1 cifra

Da `0000` a `1111`: esattamente 16 combinazioni, da 0 a F.

## 8 bit

### 2 cifre

Un **byte** si scrive sempre con due cifre: da `00` a `FF`.

## FF

### = 255

Il byte più grande: `11111111` in binario.

> Per non confonderlo con il decimale si usa un segno: `0x2F` nei programmi, `#2F` nei colori, $2F_{16}$ sul quaderno.

VALORE POSIZIONALE

# Ogni posto vale 16 volte quello a destra

| $16^3 \cdot 4096$ | $16^2 \cdot 256$ | $16^1 \cdot 16$ | $16^0 \cdot 1$ |
|---|---|---|---|
| 0 | 1 | A | 3 |

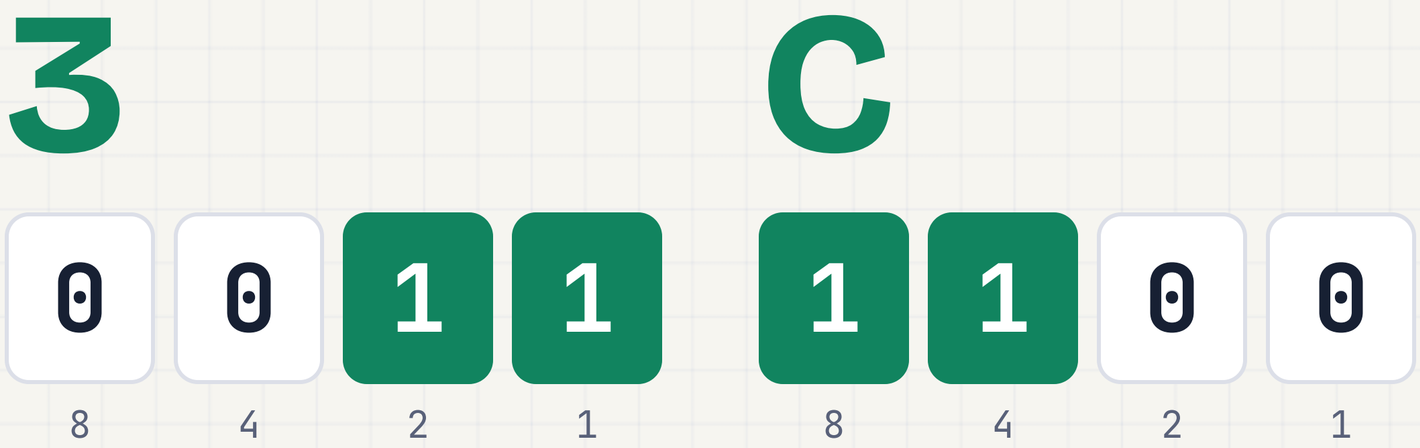
1A3 in decimale (A vale 10)

$$1 \cdot 256 + 10 \cdot 16 + 3 \cdot 1 = 419$$

Stesso metodo del binario: **cifra × peso**, poi si somma. Esempio veloce: `2F` = 2·16 + 15 = **47**.

DA ESADECIMALE A BINARIO

# Ogni cifra diventa un gruppo di 4 bit




Si affiancano i gruppi

$$3C_{16} = 0011\ 1100_2 = 60$$

DA BINARIO A ESADECIMALE

# Raggruppa a 4 bit, partendo da destra

**1**

**Il numero**

1011011

7 bit

**2**

**Gruppi da destra**

101 · 1011

l'ultimo è incompleto

**3**

**Zeri a sinistra**

0101 · 1011

ora sono 4 + 4

**4**

**Converti**

5 · B

risultato **5B**

Si parte sempre da destra: gli zeri aggiunti a sinistra non cambiano il valore.

AL COMPUTER · 15 minuti

# Prova tu: esadecimale ↔ binario

1. Apri **Calc**. In riga 1 scrivi: **Dato · A mano · Verifica**.
2. Da A2 ad A5 scrivi `7F` `A5` `1C` `B0`; in colonna B il binario a 8 bit calcolato a mano.
3. In C2 scrivi `=HEX.BINARIO(A2;8)` e trascina fino a C5.
4. Da A7 ad A9 scrivi `11110000` `101010` `11011110`; in B l'esadecimale a mano, in C `=BINARIO.HEX(A7)`.
5. Salva come **esadecimale_cognome.ods**.

### Prima a mano

La formula serve solo a controllare. Se B e C non coincidono, cerca l'errore.

**Controllo:** 7F = 01111111, 101010 = 2A.

DA DECIMALE A ESADECIMALE

# Divisioni successive per 16

1. Dividi il numero per **16**.
2. Il **resto** va da 0 a 15: scrivilo come cifra (10 = A … 15 = F).
3. Ripeti con il quoziente finché arrivi a 0.
4. Leggi i resti dal basso verso l'alto.

| Divisione | Quoziente | Resto |
|---|---|---|
| 470 : 16 | 29 | 6 |
| 29 : 16 | 1 | 13 → D |
| 1 : 16 | 0 | 1 |

Resti letti dal basso

**470 in esadecimale = 1D6**

AL COMPUTER · 15 minuti

# Le divisioni le fa Calc

| | A | B | C | D |
|---|---|---|---|---|
| 1 | Numero | Quoziente | Resto | Cifra |
| 2 | 2026 | =QUOZIENTE(A2;16) | =RESTO(A2;16) | =DECIMALE.HEX(C2) |
| 3 | =B2 | ↓ copia | ↓ copia | ↓ copia |

1. Nel file di prima aggiungi un foglio **Divisioni** e scrivi le righe 1–3.
2. Copia la riga 3 verso il basso finché il quoziente è 0. Leggi la colonna D dal basso.
3. Rifai con **300** e **1000**. Verifica con `=DECIMALE.HEX(2026)` e salva.

**Controllo:** 2026 = 7EA. Ritorno: `=HEX.DECIMALE("7EA")` dà 2026.

DOVE SI USA

# L'esadecimale è ovunque ci siano byte

### Colori

#1E90FF

In HTML e CSS: 3 byte, uno per rosso, verde e blu.

### Indirizzi MAC

3C-52-82-1A-9F-04

6 byte che identificano la scheda di rete. I primi 3 indicano il produttore.

### Memoria

0x00403A1C

Gli indirizzi delle celle di memoria, nei debugger e nei messaggi di errore.

In tutti i casi: **2 cifre esadecimali = 1 byte**.

COLORI IN HTML E CSS

# #RRGGBB: tre byte, tre luci

Ogni coppia va da 00 (spento) a FF (massimo, 255): rosso · verde · blu

`#FF8000` → 255 · 128 · 0

**#FF0000**
solo rosso

**#00FF00**
solo verde

**#0000FF**
solo blu

**#FFFFFF**
tutte al massimo: bianco

**#000000**
tutte spente: nero

**#808080**
tutte a metà: grigio

AL COMPUTER · 20 minuti

# Indovina il colore prima di vederlo

```
<style>
  div { padding: 20px; font-size: 28px; }
  .a  { background: #FFFF00; }
  .b  { background: #00FFFF; }
  .c  { background: #FF8000; }
  .d  { background: #800080; color: #FFFFFF; }
</style>
<div class="a">A</div>
<div class="b">B</div>
<div class="c">C</div>
<div class="d">D</div>
```

1. Sul quaderno scrivi la tua **previsione** per A, B, C, D.
2. Apri **Blocco note** e copia il codice.
3. Salva come **colori_cognome.html** (tipo: Tutti i file).
4. Aprilo con doppio clic nel browser e confronta.
5. Aggiungi un riquadro E con il tuo colore preferito.

**Controllo:** A è giallo, B è azzurro (ciano).

UN CENNO

# L'ottale: base 8, gruppi di 3 bit

- Cifre da **0 a 7**; ogni posto vale 8 volte quello a destra.
- 8 = $2^3$: una cifra ottale vale **3 bit**.
- Si raggruppa a 3 bit da destra, come per l'esadecimale.
- Oggi è poco usato; resiste nei **permessi dei file** su Linux.

101110 a gruppi di 3

$$101 \cdot 110 = 56_8$$

`chmod 755`

7 = 111 → leggere, scrivere, eseguire
5 = 101 → leggere, eseguire

RIPASSO

# Le quattro cose da ricordare

**1**

## Cifre e pesi

0–9 poi A–F (A = 10, F = 15). Pesi: 1, 16, 256, 4096.

**2**

## Esadecimale ↔ binario

1 cifra = 4 bit. Si raggruppa da destra.

**3**

## Decimale ↔ esadecimale

Dividi per 16 e leggi i resti dal basso; al ritorno, cifra × peso.

**4**

## Dove si usa

Colori #RRGGBB, indirizzi MAC, memoria: 2 cifre = 1 byte.